Lecture # 13

By: Muhammad Zain Uddin

email: zuddin@iba.edu.pk

IPC using SOCKETS

What are Sockets?

- **Definition:** Sockets provide a communication mechanism between processes, either on the same machine or across a network.
- **Usage:**
  - Facilitates data exchange between applications.
  - Enables communication between processes running on different machines.

Types of Sockets

1. **Unix Domain Sockets:**
    1. Communication between processes on the same machine.
    2. Identified by a file path in the file system.
    3. Supports full-duplex communication.
2. **Internet Domain Sockets:**
    1. Communication over a network.
    2. Identified by an IP address and port number.
    3. Supports various socket types (stream, datagram).

So Unix Domain Sockets and FIFO are the same?

No!

Unix domain sockets and FIFOs (named pipes) are distinct IPC mechanisms, but they share some similarities. Let's clarify:

•**Unix Domain Sockets:**

- Communication between processes on the same machine using sockets.
- Identified by a file path, often in the file system.
- Provides full-duplex communication and supports various socket types (stream, datagram).

•**FIFO (Named Pipe):**

- A named pipe is a special file used for one-way communication between processes.
- Identified by a file path (similar to Unix domain sockets in appearance).
- Typically used for simple one-way communication.

While Unix domain sockets and FIFOs share the characteristic of being identified by a file path, they serve different purposes and have different capabilities. Unix domain sockets are a more versatile IPC mechanism, providing bidirectional communication, while FIFOs are primarily designed for one-way communication.

Socket type as per data communication

1.Stream Sockets:

1. **Characteristic:**
    1. Provides a reliable, connection-oriented communication.
    2. Ensures data integrity and order of delivery.
    3. Similar to a continuous stream of data.
    4. Implemented using TCP (Transmission Control Protocol) in the Internet domain.
2. **Use Case:**
    1. Suitable for applications where ordered and error-checked delivery of data is essential.
    2. Examples include file transfers, web browsing, and email.
3. **Key Points:**
    1. Establishes a connection before data exchange.
    2. Guarantees the order of data transmission.
    3. Resends lost packets and ensures reliable delivery.

Types continue…

2. Datagram Sockets:

1. **Characteristic:**
    1. Provides connectionless, unreliable communication.
    2. Does not guarantee the order of delivery or data integrity.
    3. Operates based on discrete, independent packets (datagrams).
    4. Implemented using UDP (User Datagram Protocol) in the Internet domain.
2. **Use Case:**
    1. Suitable for real-time applications where low latency is crucial.
    2. Examples include online gaming, video streaming, and voice-over IP (VoIP).
3. **Key Points:**
    1. Does not establish a connection before data exchange.
    2. No guarantee of the order of data transmission.
    3. Do not resend lost packets, making it less reliable but faster.

Remaining two not so common types

3. Raw Sockets

•Provide direct access to lower-level protocols like **ICMP**.
•Allow development of custom protocols or deeper network interaction.
•Only accessible by superusers.
•Socket type: SOCK_RAW
•**Use Cases**: Ping tools, traceroute, network diagnostics.

4. Sequenced Packet Sockets

•Preserve record boundaries like datagram sockets but ensure ordering like stream sockets.
•Typically used in protocols like **SCTP** or **NS protocol suites**.
•Socket type: SOCK_SEQPACKET
•**Use Cases**: Specialized applications needing both sequencing and message boundaries.

Example: Server1

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <sys/un.h>

#define SOCKET_PATH "/tmp/mysocket"

int main() {
   int server_socket, client_socket;
   struct sockaddr_un server_addr, client_addr;
   socklen_t client_len = sizeof(client_addr);

   // Create a Unix domain socket
   server_socket = socket(AF_UNIX, SOCK_STREAM, 0);
   if (server_socket == -1) {
      perror("socket");
      exit(EXIT_FAILURE);
   }

   // Set up the server address structure
   memset(&server_addr, 0, sizeof(struct sockaddr_un));
   server_addr.sun_family = AF_UNIX;
   strncpy(server_addr.sun_path, SOCKET_PATH, sizeof(server_addr.sun_path) - 1);
```

```c
// Bind the socket to the address
   if (bind(server_socket, (struct sockaddr*)&server_addr, sizeof(server_addr)) == -1) {
      perror("bind");
      exit(EXIT_FAILURE);
   }

   // Listen for incoming connections
   if (listen(server_socket, 5) == -1) {
      perror("listen");
      exit(EXIT_FAILURE);
   }

   printf("Server listening on Unix domain socket...\n");

   // Accept a connection from the client
   client_socket = accept(server_socket, (struct sockaddr*)&client_addr, &client_len);
   if (client_socket == -1) {
      perror("accept");
      exit(EXIT_FAILURE);
   }

   // Send data to the client
   const char* message = "Hello from Server!";
   send(client_socket, message, strlen(message), 0);

   // Close the sockets
   close(client_socket);
   close(server_socket);

   // Remove the socket file
   unlink(SOCKET_PATH);

   return 0;
}
```

Example: Client1

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <sys/un.h>

#define SOCKET_PATH "/tmp/mysocket"

int main() {
   int client_socket;
   struct sockaddr_un server_addr;

   // Create a Unix domain socket
   client_socket = socket(AF_UNIX, SOCK_STREAM, 0);
   if (client_socket == -1) {
      perror("socket");
      exit(EXIT_FAILURE);
   }
```

```
 // Set up the server address structure
   memset(&server_addr, 0, sizeof(struct sockaddr_un));
   server_addr.sun_family = AF_UNIX;
   strncpy(server_addr.sun_path, SOCKET_PATH,
sizeof(server_addr.sun_path) - 1);

   // Connect to the server
   if (connect(client_socket, (struct sockaddr*)&server_addr,
sizeof(server_addr)) == -1) {
      perror("connect");
      exit(EXIT_FAILURE);
   }

   // Receive data from the server
   char buffer[100];
   recv(client_socket, buffer, sizeof(buffer), 0);
   printf("Client received: %s\n", buffer);

   // Close the socket
   close(client_socket);

   return 0;
}
```

So Which type of socket is this?

Unix Domain Socket

And what type of communication it does?

Stream

Example: Server2

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <string.h>
#include <arpa/inet.h>  // Add this line for inet_pton
#include <sys/types.h>
#include <sys/socket.h>
#define PORT 8080

int main() {
   int server_fd, new_socket;
   struct sockaddr_in address;
   int addrlen = sizeof(address);

   // Create socket file descriptor
   if ((server_fd = socket(AF_INET, SOCK_STREAM, 0)) == 0) {
      perror("Socket creation failed");
      exit(EXIT_FAILURE);
   }

   // Set up server address struct
   address.sin_family = AF_INET;
   inet_pton(AF_INET,"192.168.10.130", &address.sin_addr);
   address.sin_port = htons(PORT);
```

```
// Bind the socket to localhost:8080
   if (bind(server_fd, (struct sockaddr *)&address, sizeof(address)) < 0) {
      perror("Bind failed");
      exit(EXIT_FAILURE);
   }

   // Listen for incoming connections
   if (listen(server_fd, 3) < 0) {
      perror("Listen failed");
      exit(EXIT_FAILURE);
   }

   // Accept a connection
   if ((new_socket = accept(server_fd, (struct sockaddr *)&address,
(socklen_t*)&addrlen)) < 0) {
      perror("Accept failed");
      exit(EXIT_FAILURE);
   }

   // Send data to the client
   char message[] = "Hello from server!";
   send(new_socket, message, strlen(message), 0);
   printf("Message sent to client\n");

   return 0;
}
```

Example: Client2

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <string.h>
#include <netinet/in.h>

#define PORT 8080

int main() {
   int sock = 0;
   struct sockaddr_in server_address;

   // Create socket file descriptor
   if ((sock = socket(AF_INET, SOCK_STREAM, 0)) < 0) {
      perror("Socket creation error");
      exit(EXIT_FAILURE);
   }

   // Set up server address struct
   server_address.sin_family = AF_INET;
   server_address.sin_port = htons(PORT);
```

```
 // Convert IPv4 and IPv6 addresses from text to binary form
   if (inet_pton(AF_INET, "192.168.10.130", &server_address.sin_addr) <= 0)
{
      perror("Invalid address/ Address not supported");
      exit(EXIT_FAILURE);
   }

   // Connect to the server
   if (connect(sock, (struct sockaddr *)&server_address,
sizeof(server_address)) < 0) {
      perror("Connection Failed");
      exit(EXIT_FAILURE);
   }

   // Receive data from the server
   char buffer[1024] = {0};
   read(sock, buffer, sizeof(buffer));
   printf("Server message: %s\n", buffer);

   return 0;
}
```

Internet Domain Sockets – Datagram (UDP) Example

UDP Server

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#define PORT 9090
int main() {
   int sockfd;
   char buffer[1024];
   struct sockaddr_in servaddr, cliaddr;
   socklen_t len = sizeof(cliaddr);
   sockfd = socket(AF_INET, SOCK_DGRAM, 0);
   servaddr.sin_family = AF_INET;
   servaddr.sin_addr.s_addr = INADDR_ANY;
   servaddr.sin_port = htons(PORT);
   bind(sockfd, (struct sockaddr *)&servaddr, sizeof(servaddr));
   recvfrom(sockfd, buffer, sizeof(buffer), 0, (struct sockaddr *)&cliaddr, &len);
   printf("Client: %s\n", buffer);

   const char *message = "Hello from UDP Server!";
   sendto(sockfd, message, strlen(message), 0, (struct sockaddr *)&cliaddr, len);

   close(sockfd);
   return 0;
}
```

UDP Client

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#define PORT 9090
int main() {
   int sockfd;
   struct sockaddr_in servaddr;
   char buffer[1024];
   sockfd = socket(AF_INET, SOCK_DGRAM, 0);
   servaddr.sin_family = AF_INET;
   servaddr.sin_port = htons(PORT);
   inet_pton(AF_INET, "127.0.0.1", &servaddr.sin_addr);
   const char *msg = "Hello Server!";
   sendto(sockfd, msg, strlen(msg), 0, (struct sockaddr *)&servaddr,
sizeof(servaddr));
   socklen_t len = sizeof(servaddr);
   recvfrom(sockfd, buffer, sizeof(buffer), 0, (struct sockaddr *)&servaddr,
&len);
   printf("Server: %s\n", buffer);
   close(sockfd);
   return 0;
}
```

Raw Sockets – Diagnostic Tools (Ping)

Socket Type: SOCK_RAW

Access: Requires superuser privileges

Used in: ping, traceroute

// Raw sockets allow sending crafted ICMP packets

// Typically used for diagnostics, not full applications

Sequenced Packet Sockets (SOCK_SEQPACKET)

Socket Type: SOCK_SEQPACKET

Combines: Datagram boundaries + Stream ordering

Used in: SCTP (telecom protocols)

// Useful in telecom systems

// Not commonly used in general apps